1. Hardware & IoT Edge Stack (Transfer Station Segregation)

This layer handles the physical sorting of waste and data transmission from the transfer stations, aligning with your hardware documentation.

- **Microcontrollers:** ESP32-CAM and ESP32-S3 for edge processing and controlling the segregation logic.
- **Vision Sensors:** OV2940 Camera Module for continuous image capture on the conveyor belt.
- **Physical Sensors:**
    - IR/Ultrasonic proximity sensors to detect waste placement.
    - Load Cell with an HX711 amplifier to measure the weight of the incoming C&D waste.
- **Actuators & Motor Control:** Servo motors to actuate the pneumatic sorting gates, driven by TB6612 or A4888 motor drivers.
- **Connectivity:** Native Wi-Fi via the ESP32, with an optional SIM800L GSM Module for remote transfer stations lacking stable broadband.

2. AI & Machine Learning Stack

Based on **Image 2** (which shows a live object detection script running locally) and your project documents, the AI stack must support real-time video inference and predictive routing.

- **Computer Vision Frameworks:** Python and OpenCV. The interface in **Image 2** is a classic OpenCV (cv2.imshow) window running a Python script, identifying objects like "hard plastic," "timber," and "aggregate" with bounding boxes and confidence scores.
- **Model Training & Deployment:**
    - **Roboflow:** For dataset annotation, augmentation, and managing your computer vision pipelines.
    - **Edge Impulse:** For converting and deploying lightweight Convolutional Neural Network (CNN) models directly onto the ESP32 edge devices.
    - **Kaggle:** For sourcing initial training datasets for C&D materials.
- **Object Detection Algorithm:** YOLOv8 (or a similar lightweight CNN). It provides the high FPS (Frames Per Second) and accuracy required for conveyor belt sorting as seen in your 14.8 FPS readout in Image 2.
- **Fleet Routing AI:** Deep Q-Networks (DQN) and Reinforcement Learning (built with PyTorch or TensorFlow) running on the backend to dynamically solve the Capacitated Vehicle Routing Problem (CVRP) for your trucks.

3. Frontend Web Dashboard (Fleet & Station Management)

Based on the visual design language in **Images 3 through 7**, the web platform requires a modern, component-driven frontend to render data cleanly.

- **Core Framework:** React.js or Next.js. This allows for the modular card-based UI (e.g., the "Total Units" and "Revenue Generated" cards) seen across your dashboard.

- **Styling:** Tailwind CSS. The clean typography, precise padding, and color-coded status badges (Active/Idle/Maintenance) in the images strongly suggest a utility-first CSS framework.
- **Maps & Geospatial Data: Image 5** clearly displays the watermark **"Leaflet | © OpenStreetMap"** in the bottom right corner. Leaflet.js combined with OpenStreetMap tiles is perfect for rendering the live site map and plotting truck locations.
- **Data Visualization:** Chart.js or Recharts. Required to build the smooth "Weekly Collection Trend" line graphs and the "Fleet Status Distribution" doughnut charts visible in **Image 4**, as well as the "Waste by Type" bar charts in **Image 6**.

4. Customer & Contractor Mobile Application

For the application where contractors can book and track C&D waste pickups:

- **Framework:** Flutter or React Native. This allows you to build for both iOS and Android simultaneously using a single codebase.
- **Maps API:** Google Maps API or Mapbox SDK for mobile to allow contractors to drop a pin on their construction site and track the ETA of your drivers live.

5. Backend & Cloud Infrastructure

To bridge the heavy AI calculations, the IoT hardware, and the web dashboards:

- **Backend Server:** Python (Flask or FastAPI) or Node.js (Express). A Python backend is highly recommended to seamlessly integrate your existing Python-based AI object detection scripts and reinforcement learning routing algorithms.
- **Database (Relational):** PostgreSQL to store structured data like contractor accounts, vehicle registries, driver profiles, and billing information.
- **Database (Time-Series):** InfluxDB or MongoDB to handle the high-velocity stream of sensor data, load cell weights, and live GPS coordinates from the trucks.
- **Real-Time Communication:** WebSockets (Socket.io). Essential for pushing live updates to the dashboard without page refreshes, ensuring the Map pins (Image 5) and the Operations Dashboard numbers (Image 4) tick upwards in real-time as waste is processed.